

DATA 260 HOMEWORK 1

Name: Ayushi Shringi
SID4: 0036
DOMAIN_ID: 4
Domain Entity: Open-source package vulnerabilities
Port: 8036
Prefix: s0036
SEED: 0036
VERIFY_SEED: 260036
Python Version: Python 3.12.14
Model: qwen3:4b
Hardware: MacBook Pro
Chip: Apple M5 Pro
Memory: 24 GB
Operating System: macOS Tahoe 26.5.1
GitHub Repository: <https://github.com/ayushishringi/data260-0036>
Code Commit Hash: 298ce6e
Git Tag: hw1

Model choice: We were instructed to use qwen3:8b, but I used qwen3:4b because qwen3:8b was not practical to run reliably on my local 24 GB MacBook Pro configuration. I therefore used the smaller local Qwen3 model as permitted by the assignment and documented the change.

Part I and Part II: HTML and JavaScript Form

```

1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>HW1-Ayushi Shringi</title>
7      <link rel="stylesheet" href="styles.css" />
8    </head>
9    <body>
10     <main class="card">
11       <p class="eyebrow">DATA-260 · HW1 · DOMAIN_ID 4</p>
12       <h1>Vulnerability Report</h1>
13       <p class="lede">
14         File an open-source package vulnerability notice. Required fields are
15         marked.
16       </p>
17
18       <form id="vuln-form">
19         <label for="packageName">Package name</label>
20         <input
21           id="packageName"
22           name="packageName"
23           type="text"
24           required
25           autofocus
26           placeholder="Canonical package name, e.g. lodash"
27         />
28
29         <label for="affectedVersion">Affected version</label>
30         <input
31           id="affectedVersion"
32           name="affectedVersion"
33           type="text"
34           required
35           placeholder="Version or range, e.g. < 4.17.21"
36         />
37
38         <label for="submitterEmail">Submitter email</label>
39         <input
40           id="submitterEmail"
41           name="submitterEmail"
42           type="email"
43           required
44           placeholder="you@sjsu.edu"
45         />
46

```

```
<label for="submitterEmail">Submitter email</label>
<input
  id="submitterEmail"
  name="submitterEmail"
  type="email"
  required
  placeholder="you@sjsu.edu"
/>

<label for="description">Vulnerability description</label>
<textarea
  id="description"
  name="description"
  required
  rows="6"
  placeholder="Describe the issue, impact, and how it was discovered (more than 25 characters).">
</textarea>

<label for="severity">Severity</label>
<select id="severity" name="severity" required>
  <option value="" disabled selected>Select a severity</option>
  <option value="critical">Critical</option>
  <option value="high">High</option>
  <option value="medium">Medium</option>
  <option value="low">Low</option>
</select>

<div class="checkbox-row">
  <input id="agreedToTerms" name="agreedToTerms" type="checkbox" />
  <label for="agreedToTerms">I agree to the terms and conditions.</label>
</div>

<button type="submit">Report vulnerability</button>
</form>
<pre id="output" class="output" hidden></pre>
</main>
<script src="app.js"></script>
</body>
</html>
```

JS app.js > ...

```
1  const form = document.getElementById("vuln-form");
2
3  const createSubmissionCounter = () => {
4    let count = 0;
5    return () => {
6      count += 1;
7      return count;
8    };
9  };
10
11  const nextSubmissionCount = createSubmissionCounter();
12
13  const validateSubmission = ({ description, agreedToTerms }) => {
14    if (description.length <= 25) {
15      alert("Description must be more than 25 characters.");
16      return false;
17    }
18    if (!agreedToTerms) {
19      alert("You must agree to the terms and conditions.");
20      return false;
21    }
22    return true;
23  };
24
25  form.addEventListener("submit", (event) => {
26    event.preventDefault();
27
28    const formData = {
29      packageName: document.getElementById("packageName").value.trim(),
30      affectedVersion: document.getElementById("affectedVersion").value.trim(),
31      submitterEmail: document.getElementById("submitterEmail").value.trim(),
32      description: document.getElementById("description").value.trim(),
33      severity: document.getElementById("severity").value,
34      agreedToTerms: document.getElementById("agreedToTerms").checked,
35    };
36
37    if (!validateSubmission(formData)) {
38      return;
39    }
40
41    const jsonString = JSON.stringify(formData);
42    console.log("Form data as JSON string:");
43    console.log(jsonString);
44
45    const parsed = JSON.parse(jsonString);
```

```

37     if (!validateSubmission(formData)) {
38         return;
39     }
40
41     const jsonString = JSON.stringify(formData);
42     console.log("Form data as JSON string:");
43     console.log(jsonString);
44
45     const parsed = JSON.parse(jsonString);
46     const { packageName, submitterEmail } = parsed;
47     console.log("Primary field (packageName):", packageName);
48     console.log("Email field (submitterEmail):", submitterEmail);
49
50     const withDate = {
51         ...parsed,
52         submissionDate: new Date().toISOString(),
53     };
54     console.log("Parsed object with submissionDate:");
55     console.log(withDate);
56
57     const submissionCount = nextSubmissionCount();
58     console.log("Successful submission count:", submissionCount);
59
60     const output = document.getElementById("output");
61     output.hidden = false;
62     output.textContent = [
63         "JSON string:",
64         jsonString,
65         "",
66         "Primary field (packageName): " + packageName,
67         "Email field (submitterEmail): " + submitterEmail,
68         "",
69         "Object with submissionDate:",
70         JSON.stringify(withDate, null, 2),
71         "",
72         "Successful submission count: " + submissionCount,
73     ].join("\n");
74
75     alert(`Vulnerability report submitted (${submissionCount}). Check the console (and the output box) for JSON.`);
76 }
77

```

Required-field validation -

127.0.0.1:8036/index.html

HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name Fill out this field

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

Description Length Verification-

127.0.0.1:8036/index.html

HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

lodash

Affected version

3.8

Submitter email

tes@g

Vulnerability description

affected- high

Severity

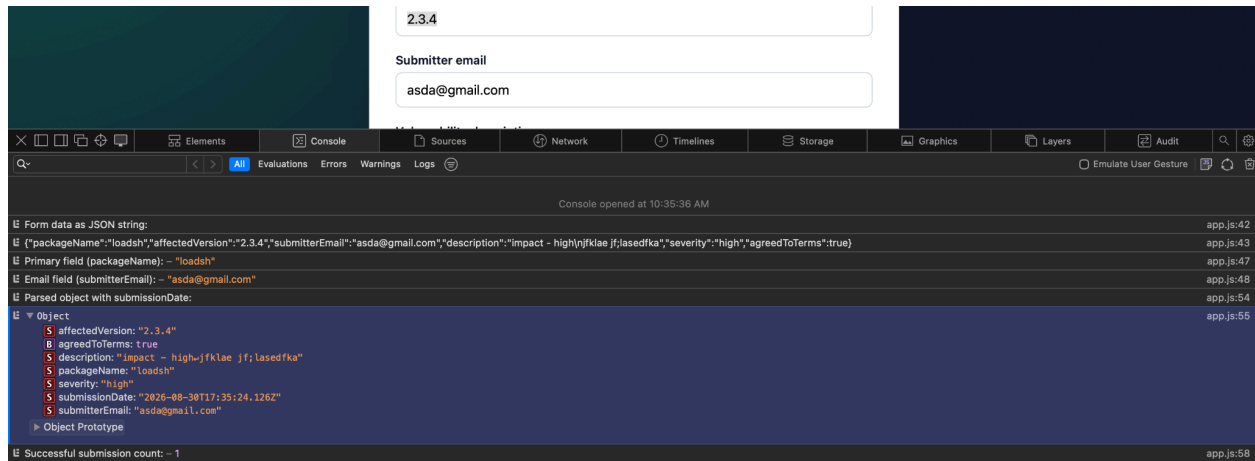
Medium

☐ I agree to the terms and conditions.

Report vulnerability

Description must be more than 25 characters. [Close](#)

Console Logging-



Terms-and-Conditions Validation -

127.0.0.1:8036/index.html
HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Affected version

Submitter email

Vulnerability description

affected- high
kekfak jdnfkadnf kankndnfkla dgan kakkfkang ka

Severity

☐ I agree to the terms and conditions.

[Close](#)

Report vulnerability

Part III: Docker Containerization

Docker container running locally-

```

● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
RTS
30f2708a07f3   data260-0036-hw1:latest             "/docker-entrypoint..." 13 seconds ago Up 12 seconds 0.
0.0.0:8036->8036/tcp, [::]:8036->8036/tcp data260-0036-vuln-form-1
○ ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %

```

From running through Docker-

127.0.0.1:8036/index.html

HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

ECS Service-

data260-0036-service has been deployed successfully.

View service

data260-0036-cluster

Last updated August 30, 2026, 14:15 (UTC-7:00)

Actions

Create with Express Mode

Cluster overview

Cluster ARN

arn:aws:ecs:us-west-2:232025405374:cluster/da...

Status

Active

CloudWatch monitoring

Default

Action logs

Turned off

Services

0 active | 0 draining

Tasks

0 running | 0 pending

Registered container instances

-

Services

Tasks

Daemons

Infrastructure

Metrics

Scheduled tasks

Configuration - updated

Event history

Tags

Services (1) info

Last updated August 30, 2026, 14:30 (UTC-7:00)

Manage tags

Update

Delete service

Create

Filter services by value

Filter launch type Any launch type

Filter scheduling strategy Any scheduling strategy

Filter resource management type Any resource management type

< 1 >

☐	Service name	ARN	Status	Schedu...	Launc...	Task de...	Deployments and tasks	Last deployment	Last depla
☐	data260-0036-service	arn:awssec...	Active	Replica	Fargate	data260-...	1/1 Tasks running	Completed	View 17 minutes

Configuration

Operating system/Architecture: Linux/ARM64

Capacity provider: -

Platform version: 1.4.0

Task role: -

Task group: service:data260-0036-service | [View service](#)

ECS Exec: [Info](#)

Task execution role: [ecsTaskExecutionRole](#)

Fault injection: ☐ Turned off

Fargate ephemeral storage

Encryption: [Info](#)
Default AWS Fargate encryption

Size (GiB): 20

Networking

Network mode: awsvpc

ENI ID: [eni-0f56eac57276f9e65](#)

Subnet: [subnet-04d28f0566eace32](#)

Security group: [sg-08b29299d412da2c5 \(data260-0036-sg\)](#)

Private IP: [172.31.16.122](#)

Public IP: [44.247.157.122](#) | [open address](#)

IPv6 address: -

MAC address: [02:ff:da:a7:d9:5d](#)

Task scale-in protection

Protection status: ☐ Turned off

Protection duration: -

Containers (1)

Filter containers

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU	Memory hard/soft limit
vuln-form	24923e973a4e4d...	docker.io/...	sha256:f8...	Running	Unknown	0	- / -

Task Details-

data260-0036-service has been deployed successfully. [View service](#)

24923e973a4e4d74bc5e8029c4cab2e5

Last updated: August 30, 2026, 14:31 (UTC-7:00) [Refresh](#) [Stop](#)

Configuration | Metrics | Logs | Volumes (0) | Tags

Task overview

Task ID: [24923e973a4e4d74bc5e8029c4cab2e5](#)

Last status: [Running](#)

Task ARN: [arn:aws:ecs:us-west-2:232025405374:task/dat...](#)

Desired status: [Running](#)

Task definition: revision [data260-0036-hw1.1](#)

Service: [data260-0036-service](#)

CPU | memory: .25 vCPU | .5 GiB

Service revision: [1935375451536166861](#)

Launch type: Fargate

Started by: [ecs-svc/1935375451536166861](#)

Task lifecycle

- Created: August 30, 2026, 14:14 (UTC-7:00)
- Provisioning
 - Network interface attached: August 30, 2026, 14:14 (UTC-7:00)
- Pending
 - Pull started: August 30, 2026, 14:14 (UTC-7:00)
 - Pull completed: August 30, 2026, 14:14 (UTC-7:00) (5 seconds)
- Running: August 30, 2026, 14:14 (UTC-7:00)
Duration: 17 minutes, 46 seconds

Configuration

Operating system/Architecture: Linux/ARM64

Capacity provider: -

Platform version: 1.4.0

Task role: -

Task group: service:data260-0036-service | [View service](#)

ECS Exec: [Info](#)

Task execution role: [ecsTaskExecutionRole](#)

Fault injection: ☐ Turned off

Container details for vuln-form

Application Deployed on AWS-

Task configuration | Elastic Container Service | us-west-2

HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

Planner–Reviewer–Finalizer agent pipeline-

```

1 def run_pipeline(title: str, content: str, temperature: float, client: ModelClient) -> dict:
2     source = f"Title: {title}\n\nBody:\n\n{content}"
3
4     planner = client.complete(
5         [
6             {"role": "system", "content": PLANNER_SYSTEM},
7             {"role": "user", "content": source},
8         ],
9         temperature=temperature,
10        format=TAGS_SUMMARY_SCHEMA,
11    )
12    planner_json = _extract_json(planner.text)
13    if not _has_tags(planner_json):
14        planner = client.complete(
15            [
16                {"role": "system", "content": PLANNER_SYSTEM},
17                {
18                    "role": "user",
19                    "content": source + "\n\nYour previous reply was invalid. Return tags[3] and summary only."
20                },
21            ],
22            temperature=0.0,
23            format=TAGS_SUMMARY_SCHEMA,
24        )
25        planner_json = _extract_json(planner.text)
26
27    reviewer = client.complete(
28        [
29            {"role": "system", "content": REVIEWER_SYSTEM},
30            {
31                "role": "user",
32                "content": (
33                    source
34                    + "\n\nPlanner JSON:\n"
35                    + json.dumps(planner_json)
36                ),
37            },
38        ],
39        temperature=temperature,
40        format=REVIEWER_SCHEMA,
41    )
42    reviewer_json = _extract_json(reviewer.text)
43    if not _has_tags(reviewer_json):

```

```

168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
176     if len(planner_summary.split()) <= 25:
177         reviewer_json["summary"] = planner_summary
178
179     actual_changed = (
180         reviewer_json.get("tags", []) != planner_tags
181         or reviewer_json.get("summary", "") != planner_summary
182     )
183     reviewer_json["changed"] = actual_changed
184
185     if not actual_changed:
186         reviewer_json["notes"] = (
187             "No changes required; tags and summary are valid."
188         )
189
190     finalizer = client.complete(
191         [
192             {"role": "system", "content": FINALIZER_SYSTEM},
193             {
194                 "role": "user",
195                 "content": json.dumps(
196                     {"planner": planner_json, "reviewer": reviewer_json}
197                 ),
198             },
199         ],
200         temperature=temperature,
201         format=PUBLISH_SCHEMA,
202     )
203     publish = _normalize_publish(_extract_json(finalizer.text))
204
205     return {
206         "planner": planner_json,
207         "planner_raw": planner.text,
208         "reviewer": reviewer_json,
209         "reviewer_raw": reviewer.text,
210         "publish": publish,
211         "tokens": {

```

```

221 parser.add_argument("--title")
222 parser.add_argument("--content")
223 parser.add_argument("--input", type=Path, help="JSON with title and content")
224 parser.add_argument("--temperature", type=float, default=0.2)
225 parser.add_argument("--model", default=None)
226 parser.add_argument("--repeat", type=int, default=1)
227 parser.add_argument("--out", type=Path, default=None)
228 args = parser.parse_args()
229
230 if args.input:
231     payload = json.loads(args.input.read_text())
232     title = payload["title"]
233     content = payload["content"]
234 elif args.title and args.content:
235     title, content = args.title, args.content
236 else:
237     parser.error("Provide --title and --content, or --input JSON")
238
239 client = ModelClient(model=args.model) if args.model else ModelClient()
240 runs = []
241 for i in range(args.repeat):
242     started = time.perf_counter()
243     result = run_pipeline(title, content, args.temperature, client)
244     elapsed_ms = (time.perf_counter() - started) * 1000
245     record = {
246         "run": i + 1,
247         "temperature": args.temperature,
248         "latency_ms": round(elapsed_ms, 2),
249         "tags": result["publish"]["tags"],
250         "summary": result["publish"]["summary"],
251         "planner": result["planner"],
252         "reviewer": result["reviewer"],
253         "publish": result["publish"],
254     }
255     runs.append(record)
256     print("=" * 72)
257     print(f"RUN {i + 1} temp={args.temperature} latency_ms={elapsed_ms:.1f}")
258     print("Planner output:")
259     print(json.dumps(result["planner"], indent=2))
260     print("Reviewer output:")
261     print(json.dumps(result["reviewer"], indent=2))
262     print("Finalized / Publish output:")
263     print(json.dumps(result["publish"], indent=2))
264

```

The screenshot shows a VS Code editor with a file explorer on the left containing files like .gitignore, AGENT.md, agents_demo.py, app.js, docker-compose.yml, Dockerfile, DOMAIN_SCHEMA.md, hw1_client.py, index.html, Makefile, nginx.conf, README.md, and styles.css. The main editor displays a Python script with the following code:

```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
```

The terminal output shows the command `python3 agents_demo.py --input reports/hw01/cases/nondeterminism_input.json` and its output:

```
RUN 1 temp=0.2 latency_ms=5086.0
Planner output:
{
  "tags": [
    "grpc-security",
    "protobuf-vulnerability",
    "network-privilege-escalation"
  ],
  "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads.",
  "notes": "Tags focus on security implications, specific vulnerability scope, and attack vector. Summary concisely captures the vulnerability's impact and mitigation."
}
Reviewer output:
{
  "tags": [
    "grpc-security",
    "protobuf-vulnerability",
    "network-privilege-escalation"
  ],
  "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads.",
  "changed": false,
  "notes": "No changes required; tags and summary are valid."
}
Finalized / Publish output:
{
  "tags": [
    "grpc-security",
    "protobuf-vulnerability",
    "network-privilege-escalation"
  ],
  "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads."
}
```

Agentic AI Pipeline

Forty-run nondeterminism experiment -

The screenshot shows a VS Code editor with a file explorer on the left containing files like .gitignore, AGENT.md, agents_demo.py, app.js, docker-compose.yml, Dockerfile, DOMAIN_SCHEMA.md, hw1_client.py, index.html, Makefile, nginx.conf, README.md, and styles.css. The main editor displays a Python script with the following code:

```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
```

The terminal output shows the command `python -c 'import csv,json,sys,collections; rows=list(csv.DictReader(open("reports/hw01/raw/nondeterminism_40_runs.csv"))); keys=[k for k in rows[0] if k.lower() in ("temperature","temp")]; counts=collections.Counter(str(r[key]) for r in rows); metrics=json.load(open("reports/hw01/raw/nondeterminism_metrics.json")); print("Python version:",sys.version.split()[0]); print("Total runs:",len(rows)); print("Temperature 0.0 runs:",counts["0.0"]); print("Temperature 0.7 runs:",counts["0.7"]); print("Temp 0.0 distinct tag sets:",metrics["0.0"]["distinct_tag_sets"]); print("Temp 0.7 distinct tag sets:",metrics["0.7"]["distinct_tag_sets"]); print("Result: temperature 0.7 produced greater output variability.")'` and its output:

```
Total runs: 40
Temperature 0.0 runs: 20
Temperature 0.7 runs: 20
Temp 0.0 distinct tag sets: 1
Temp 0.7 distinct tag sets: 18
Result: temperature 0.7 produced greater output variability.
```

Five-Turn Client and Token Accounting-

```

src > model_client.py > ModelClient > complete
1  """Stable Ollama chat adapter used by every HW1 model call."""
2
3  from __future__ import annotations
4
5  import json
6  import os
7  import re
8  import urllib.error
9  import urllib.request
10 from dataclasses import dataclass, field
11 from typing import Any, Optional
12
13
14 DEFAULT_MODEL = os.environ.get("OLLAMA_MODEL", "qwen3:4b")
15 DEFAULT_HOST = os.environ.get("OLLAMA_HOST", "http://127.0.0.1:11434")
16 THINK_RE = re.compile(
17     r"^\s*(?:<think>)?.*?</think>\s*",
18     re.DOTALL | re.IGNORECASE,
19 )
20
21
22 @dataclass
23 class CompletionResult:
24     text: str
25     input_tokens: int
26     output_tokens: int
27     total_tokens: int
28     raw: dict[str, Any] = field(default_factory=dict)
29
30
31 class ModelClient:
32     def __init__(self, model: str = DEFAULT_MODEL, host: str = DEFAULT_HOST) -> None:
33         self.model = model
34         self.host = host.rstrip("/")
35
36     def complete(
37         self,
38         messages: list[dict[str, str]],
39         tools: Optional[list[dict[str, Any]]] = None,
40         temperature: float = 0.2,
41         format: Optional[str | dict[str, Any]] = None,
42     ) -> CompletionResult:
43         payload: dict[str, Any] = {
44             "model": self.model,
45             "messages": messages,
46             "stream": False,
47             "think": False,
48             "options": {"temperature": temperature},
49         }
50         if tools:
51             payload["tools"] = tools
52         if format is not None:
53             payload["format"] = format
54
55         data = json.dumps(payload).encode("utf-8")

```



```

55     data = json.dumps(payload).encode("utf-8")
56     req = urllib.request.Request(
57         f"{self.host}/api/chat",
58         data=data,
59         headers={"Content-Type": "application/json"},
60         method="POST",
61     )
62     try:
63         with urllib.request.urlopen(req, timeout=180) as resp:
64             body = json.loads(resp.read().decode("utf-8"))
65     except urllib.error.URLError as exc:
66         raise RuntimeError(
67             f"Ollama request failed at {self.host}. Is `ollama serve` running? {exc}"
68         ) from exc
69
70     message = body.get("message") or {}
71     text = THINK_RE.sub("", message.get("content") or "").strip()
72     input_tokens = int(body.get("prompt_eval_count") or 0)
73     output_tokens = int(body.get("eval_count") or 0)
74     return CompletionResult(
75         text=text.strip(),
76         input_tokens=input_tokens,
77         output_tokens=output_tokens,
78         total_tokens=input_tokens + output_tokens,
79         raw=body,
80     )
81
82
83 def complete(
84     messages: list[dict[str, str]],
85     tools: Optional[list[dict[str, Any]]] = None,
86     temperature: float = 0.2,
87     format: Optional[str | dict[str, Any]] = None,
88     client: Optional[ModelClient] = None,
89 ) -> CompletionResult:
90     adapter = client or ModelClient()
91     return adapter.complete(
92         messages, tools=tools, temperature=temperature, format=format
93     )
94

```

```

((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 -u hw1_client.py | tee reports/
hw01/raw/hw1_client_console.txt

=== turn 1 user ===
Review this function: def add(a,b): return a+b
=== turn 1 assistant ===
- The function correctly computes the sum of two numeric inputs (if they are numbers).
- The function has no security vulnerabilities as it does not interact with external systems.
[tokens] input=106 output=3273 total=3379

=== turn 2 user ===
The function does not check types. What would you change?
=== turn 2 assistant ===
- Add type validation to ensure both a and b are numeric (e.g., integers or floats) before returning
their sum.
[tokens] input=161 output=1522 total=1683

=== turn 3 user ===
Rewrite the review for a version that adds logging of arguments.
=== turn 3 assistant ===
- The function correctly logs the arguments before returning their sum, improving debuggability.
- The function has no security vulnerabilities as it does not interact with external systems.
[tokens] input=209 output=4275 total=4484
--- /stats ---
turn_count: 3
cumulative_input_tokens: 476
cumulative_output_tokens: 9070
cumulative_total_tokens: 9546
serialized_conversation_history_length: 1272
=====

=== turn 4 user ===
Is logging user-controlled strings a security issue here?
=== turn 4 assistant ===
- Logging user-controlled strings can expose sensitive information if the logs are accessible to unau-
thorized users, posing a security risk.
- The function does not introduce new security vulnerabilities beyond the logging mechanism itself.
[tokens] input=262 output=3173 total=3435

=== turn 5 user ===
Give a final bullet-only verdict on shipping add().
=== turn 5 assistant ===
- The function should not be shipped as logging user-controlled strings could expose sensitive inform-
ation if logs are accessible to unauthorized users.
[tokens] input=321 output=1076 total=1397
--- /stats ---
turn_count: 5
cumulative_input_tokens: 1059
cumulative_output_tokens: 13319
cumulative_total_tokens: 14378
serialized_conversation_history_length: 1917
=====

=== exit totals ===
cumulative_input_tokens: 1059
cumulative_output_tokens: 13319
cumulative_total_tokens: 14378
turn_count: 5
machine_readable_tokens: reports/hw01/raw/hw1_client_tokens.json

```

Nondeterminism Results

Temperature 0.0

Distinct tag sets: 1

Tags appearing in all 20 runs: grpc-security, network-using-privilege-escalation, protobuf-vulnerability

Tags appearing in exactly one run: None

Latency p50 / p95 / p99: 3716.2 / 3934.1 / 3947.6 ms

Temperature 0.7

Distinct tag sets: 18

Tags appearing in all 20 runs: None

Tags appearing in exactly one run: gRPC; grpc; grpc-security-vulnerabilities; network reflection; network reflection leaks; network vulnerability; network-exposure; network-isolation; network-privilege; network-privilege-escalation; network-reflect:leak; network-reflectable-leaks; network-reflective-attack; network-reflective-leak; network-reflective-leaks; network-using; network-vulnerability; protobuf; protobuf reflection; protobuf vulnerabilities; protobuf-java; protobuf-java vulnerabilities; protobuf-reflect; protobuf-reflect-uses; protobuf-upgrades; reflection-leaks; security-vulnerability; security_vulnerability

Latency p50 / p95 / p99: 3269.9 / 4337.7 / 4449.2 ms

Two users submitting identical input at temperature 0.7 could receive different tag combinations, while temperature 0.0 produced the same tag set in all 20 runs.

Acceptable variation: suggested related-topic tags on an advisory page.

Unacceptable variation: using generated tags for vulnerability severity or ticket routing because identical reports could be classified differently.

Machine-readable five-turn client verification-

```
● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 -c 'import json; d=json.load(open("reports/hw01/raw/hw1_client_tokens.json")); bullet_ok=all(all(line.strip().startswith("- ") for line in t["assistant"].splitlines() if line.strip()) for t in d["turns"]); print("Python:",d["python_version"]); print("Model:",d["model"]); print("Turns:",len(d["turns"])); print("Stats after turns:",[s["after_turn"] for s in d["stats"]]); print("Totals:",d["totals"]); print("Bullet-only compliance:",bullet_ok)'\nPython: 3.12.14\nModel: qwen3:4b\nTurns: 5\nStats after turns: [3, 5]\nTotals: {'turn_count': 5, 'cumulative_input_tokens': 1059, 'cumulative_output_tokens': 13319, 'cumulative_total_tokens': 14378}\nBullet-only compliance: True
```

Final automated verification-

```

● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 scripts/verify_hw01.py
{
  "passed": true,
  "failed_files": [],
  "checks": {
    "python_version": "3.12.14",
    "python_major_minor": "3.12",
    "config": {
      "SID4": "0036",
      "PORT_BASE": 8036,
      "PREFIX": "s0036",
      "SEED": "0036",
      "VERIFY_SEED": 260036,
      "DOMAIN_ID": 4,
      "assigned_domain": "Open-source package vulnerabilities"
    },
    "files": {
      "index.html": true,
      "app.js": true,
      "styles.css": true,
      "DOMAIN_SCHEMA.md": true,
      "AGENT.md": true,
      "agents_demo.py": true,
      "hw1_client.py": true,
      "src/model_client.py": true,
      "Dockerfile": true,
      "docker-compose.yml": true,
      "Makefile": true,
      "README.md": true,
      "reports/hw01/cases/nondeterminism_input.json": true,
      "reports/hw01/AI_USE.md": true,
      "reports/hw01/METRICS.md": true
    },
    "html_title": true,
    "h1_entity": true,
    "schema_matches_domain": true,
    "localhost_port_open": true,
    "ollama_reachable": true,
    "ollama_models": "qwen3:4b",
    "timestamp_utc": "2026-08-31T16:46:23.767659+00:00"
  }
}
● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %

```

AI Use - HW1

1. ****What AI assistants were used for versus what I did.****

I used ChatGPT/Codex to help draft and review the Ollama model adapter, Planner-Reviewer-Finalizer pipeline, verification scripts, and report structure. I reviewed the generated code, executed every command, tested the form, ran Docker and Ollama locally, completed the 40-run nondeterminism experiment, and collected the report evidence.

2. ****AI-produced output that was wrong or unsuitable.****

The Qwen3 Reviewer sometimes incorrectly changed the valid version range `3.21-3.25` to forms such as `3.21-3:25`. Additionally, Ollama sometimes returned internal reasoning text even though the request specified `think: false`.

3. ****How the problems were detected and verified.****

I detected the version-range error by comparing the Planner, Reviewer, and Finalizer console outputs. I detected the reasoning-text problem through a direct Ollama API test and unusually large output-token counts. I reran the agent pipeline and five-turn client after making corrections and verified the saved machine-readable results.

4. ****What changed and why it works now.****

I added deterministic validation that preserves a valid Planner summary when it is already within the 25-word limit and accurately calculates the Reviewer's `changed` value. I also updated `src/model\_client.py` to remove reasoning text ending in `<</think>` before returning model content. The project now runs in a Python `3.12.14` virtual environment, produces clean bullet-only responses, records per-turn and cumulative token counts, and passes `scripts/verify\_hw01.py`.

Short Answers - Agentic Pipeline

Q1. The 3 final tags:

grpc-security, protobuf-vulnerability, network-privilege-escalation

Q2. The final summary(22 words):

"gRPC reflection in protobuf-java 3.21-3.25 leaks private field names across package versions, enabling attackers to map internal authorization flags via error payloads."

Q3. Did the Reviewer agent change anything? No.

The Reviewer returned the same three tags and summary as the Planner. Therefore, the change was false and the notes correctly stated that no changes were required.

Explanation of each step (in my own words):

- Planner reads the raw title/body cold and invents 3 tags plus a draft summary, with no awareness of any prior turn.
- Reviewer receives the original text plus the Planner's JSON, checks each tag actually appears as a concept in the source, verifies the summary is at most 25 words and factually unchanged (especially version numbers), and sets changed=true/false accordingly.
- Finalizer reconciles Planner + Reviewer into one publish JSON (exactly 3 tags, ≤25-word summary), preferring the Reviewer's values when valid,

In this successful run, the Reviewer made no changes, and the Finalizer published the validated tags and summary..

Part 4 - Conceptual Questions

Why is prior conversation context resent with every turn?

Stateless chat APIs do not store your session. Each `complete()` call sends the full messages list so the model can see prior turns.

How is a system prompt different from a user message?

A system prompt is a standing instruction (here: bullet-only code review, from AGENT.md). A user message is the current request in the conversation.

Why do input tokens grow over a conversation?

Because every later turn resends all earlier messages input tokens for turn N include the entire history up through turn N-1, not just the new question.

What eventually limits that growth?

The model's context window. Beyond that, any truncation or summarization strategy added in the client (this implementation does neither, so a long enough conversation would eventually exceed qwen3:4b's context limit and fail or get silently truncated by Ollama).